BÀI 01: TỔNG QUAN VỀ NGÔN NGỮ LẬP TRÌNH DART

MỤC TIÊU

- Giới thiệu tổng quan về lập trình di động
- Tổng quan về hệ điều hành android
- Giới thiệu tổng quan về ngôn ngữ lập trình Dart

1.1. Tổng quan về lập trình di động

Lập trình di động (Mobile Application Development) tập trung vào việc thiết kế, xây dựng và triển khai các ứng dụng chạy trên thiết bị di động: điện thoại thông minh và máy tính bảng. Điểm đặc trưng đối với ứng dụng di động là khả năng đáp ứng các yêu cầu đặc thù về giao diện cảm ứng, tài nguyên phần cứng hạn chế và khả năng thích nghi với nhiều kích thước màn hình, phiên bản hệ điều hành khác nhau.

Android và iOS là hai hệ điều hành di động phổ biến nhất hiện nay chiếm lĩnh phần lớn thị trường thiết bị thông minh trên toàn cầu. Android là hệ điều hành mã nguồn mở, được phát triển theo hướng linh hoạt, cho phép chạy trên nhiều dòng thiết bị và phần cứng khác nhau, đồng thời hỗ trợ đa dạng công cụ và ngôn ngữ lập trình. iOS là hệ điều hành mã nguồn đóng, được phát triển đồng bộ giữa phần mềm và phần cứng, nổi bật với tính ổn định cao, hiệu năng tối ưu và trải nghiệm người dùng nhất quán. Sự khác biệt về kiến trúc, công cụ phát triển và hệ sinh thái của Android và iOS đã hình thành hai hướng tiếp cận riêng trong lập trình di động.

1.2. Tổng quan về hệ điều hành android

1.2.1 Lịch sử hình thành và phát triển

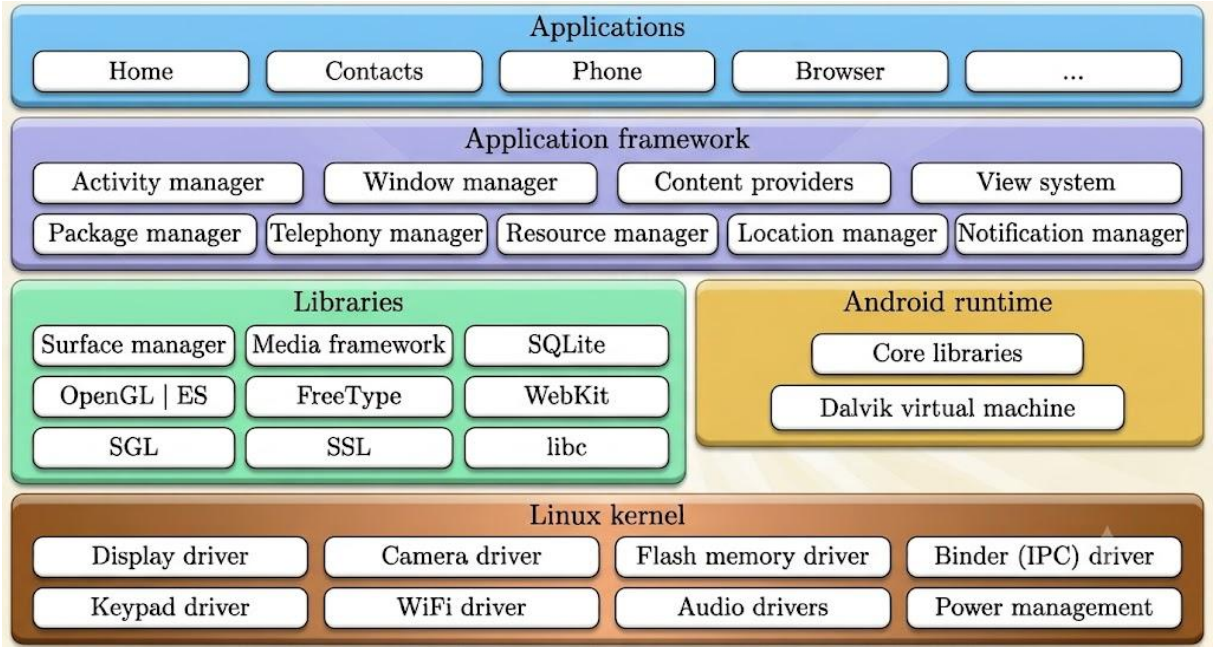
Android là một hệ điều hành dựa trên nền tảng Linux được thiết kế dành cho các thiết bị di động có màn hình cảm ứng như điện thoại thông minh và máy tính bảng. Android được phát triển bởi Android Inc., được thành lập tại Palo Alto, California vào tháng 10 năm 2003 với sự hỗ trợ tài chính từ Google và sau này được chính Google mua lại vào năm 2005.

Android ra mắt vào năm 2007 cùng với tuyên bố thành lập Liên minh thiết bị cầm tay mở: một hiệp hội gồm các công ty phần cứng, phần mềm, và viễn thông với mục tiêu đẩy mạnh các tiêu chuẩn mở cho các thiết bị di động.

- 2008: Phiên bản Android 1.0 ra mắt cùng chiếc điện thoại HTC Dream.
- 2011: Android trở thành hệ điều hành di động phổ biến nhất thế giới.
- 2014: Android Lollipop giới thiệu giao diện Material Design.
- 2021: Android 12 với giao diện Material You, cải tiến về quyền riêng tư.
- 2022: Android 13, tùy chỉnh giao diện nâng cao, bảo mật tốt hơn.
- 2023: Android 14, cải thiện quyền riêng tư và tối ưu hóa cho màn hình gập.

− 2024: Android 15, tăng cường bảo mật & quyền riêng tư.

1.2.2 Kiến trúc android



Kiến trúc Android gồm **5** tầng chính:

– Linux Kernel
– Android Runtime (ART)
– Android Libraries
– Application Framework
– Applications

1.2.2.1 Linux Kernel

Linux Kernel là lớp nền tảng thấp nhất trong kiến trúc Android. Android sử dụng nhân Linux đã được tùy biến để phù hợp với môi trường thiết bị di động, giúp đảm bảo **hiệu năng và độ ổn định** cho toàn bộ hệ thống. Lớp Linux Kernel chịu trách nhiệm:

– Quản lý tiến trình (process management)
– Quản lý bộ nhớ (memory management)
– Quản lý thiết bị phần cứng (drivers)
– Bảo mật và cấp quyền truy cập hệ thống

1.2.2.2 Android Runtime (ART)

Android Runtime (ART) là môi trường thực thi các ứng dụng Android. ART sử dụng cơ chế **Ahead-of-Time (AOT)** và **Just-in-Time (JIT)** compilation để cải thiện tốc độ thực thi so với các phiên bản Android cũ. Đây là thành phần quan trọng giúp ứng dụng Android hoạt động mượt mà trên các thiết bị có tài nguyên hạn chế. ART có nhiệm vụ:

Thực thi mã ứng dụng (Java/Kotlin đã được biên dịch)
Quản lý bộ nhớ và thu gom rác (Garbage Collection)
Tối ưu hiệu năng khi chạy ứng dụng

1.2.2.3. Android Libraries

Android Libraries cung cấp tập hợp các thư viện hệ thống, được truy cập thông qua tầng Application Framework, hỗ trợ các chức năng cốt lõi cho hệ điều hành và ứng dụng, bao gồm:

- Thư viện đồ họa (OpenGL, Skia)
- Thư viện xử lý media (audio, video)
- Thư viện cơ sở dữ liệu (SQLite)
- Thư viện mạng và bảo mật

1.2.2.4. Application Framework

Application Framework là tầng cung cấp các API cho phép xây dựng ứng dụng và tương tác với hệ thống, bao gồm:

- Activity Manager: quản lý vòng đời Activity
- Window Manager: quản lý cửa sổ giao diện
- Content Providers: chia sẻ và truy cập dữ liệu
- View System: xây dựng giao diện người dùng
- Resource Manager: quản lý tài nguyên (layout, image, string, …)

1.2.2.5. Applications

Applications là tầng trên cùng trong kiến trúc Android, bao gồm:

- Các ứng dụng hệ thống (Phone, Contacts, Settings, …)
- Các ứng dụng do người dùng cài đặt

Mỗi ứng dụng Android chạy trong một tiến trình riêng và được cô lập với các ứng dụng khác nhằm đảm bảo an toàn và bảo mật. Ứng dụng được xây dựng từ các thành phần chính như:

- Activity
- Fragment
- Service
- Broadcast Receiver
- Content Provider

1.3 Ngôn ngữ và công cụ phát triển ứng dụng di động

Ứng dụng di động được phát triển bằng các ngôn ngữ và công cụ chuyên biệt theo từng nền tảng

- **Android:** Java, Kotlin – Android Studio, SDK, Emulator
- **iOS:** Swift, Objective-C – Xcode, Simulator
- **Đa nền tảng:** Flutter, React Native – chia sẻ mã nguồn
- Công cụ phát triển hỗ trợ **thiết kế giao diện, lập trình, kiểm thử và triển khai ứng dụng**

1.4 Tổng quan về ngôn ngữ Dart

Dart là một ngôn ngữ lập trình hiện đại, mã nguồn mở, do **Google** phát triển, được thiết kế để xây dựng **ứng dụng đa nền tảng**, đặc biệt là các ứng dụng di động với **Flutter**, nhưng cũng có thể được sử dụng để phát triển web, desktop, backend server và các ứng dụng nhúng.

Dart được sử dụng phổ biến nhất trong **Flutter**, một framework phát triển ứng dụng UI của Google.

1.4.1. Đặc điểm chính của Dart

– Đa nền tảng: Dart có thể được sử dụng để phát triển ứng dụng mobile (Android, iOS), web, desktop và server từ một codebase duy nhất.

– Hiệu suất cao: Dart hỗ trợ biên dịch Ahead-of-Time (AOT), giúp ứng dụng chạy nhanh hơn đồng thời cũng hỗ trợ biên dịch Just-in-Time (JIT), giúp tăng tốc độ phát triển nhờ Hot Reload trong Flutter.

– Cú pháp dễ học, dễ gần: Dart có cú pháp tương tự JavaScript, Java, C#, dễ dàng tiếp cận với những người đã có nền tảng về các ngôn ngữ này.

– Là ngôn ngữ lập trình hướng đối tượng (OOP): Hỗ trợ class, object, interface, mixins, async/await, giúp tổ chức code tốt hơn.

– Hỗ trợ lập trình bất đồng bộ (Asynchronous): Dart sử dụng Future & Stream để xử lý các tác vụ bất đồng bộ, giúp viết code gọn gàng hơn.

1.4.2. Cấu trúc cơ bản của một chương trình Dart

Giống như lập trình C/C++ chương trình chính của Dart được bắt đầu từ hàm ***main***().
Ví dụ:

```
void main() {   print("Chào mừng bạn đến với Dart!"); }
```

1.4.2.1. Kiểu dữ liệu của Dart

Kiểu dữ liệu	Mô tả	Ví dụ
int	Số nguyên	int x = 10;
double	Số thực (dấu phẩy động)	double y = 3.14;
String	Chuỗi ký tự	String name = "Dart";
bool	Boolean (true/false)	bool isValid = true;
List	Danh sách (mảng)	List<int> numbers = [1, 2, 3];
Set	Tập hợp không trùng lặp	Set<String> unique = {"A", "B", "C"};

Map	Bản đồ (key-value)	Map<String, int> scores = {"Alice": 95};
null	Giá trị rỗng	String? nullable;
dynamic	Kiểu động, có thể thay đổi	dynamic x = "Hello"; x = 10;
var	Suy luận kiểu (không thay đổi)	var name = "Dart";
Object	Kiểu dữ liệu cha của tất cả	Object x = 10;

1.4.2.2 Biến

Trong **Dart**, biến là một phần quan trọng giúp lưu trữ và quản lý dữ liệu. Dart hỗ trợ nhiều loại biến với các quy tắc rõ ràng về kiểu dữ liệu, phạm vi và cách sử dụng.

Khai báo biến trong Dart

Có thể khai báo biến giống ngôn ngữ lập trình C/C++ theo cú pháp:

Kiểu_dữ_liệu tên_biến [=giá trị];

Ví dụ: int x=10;

String name=”An”;

Có thể khai báo biến có thể nhận giá trị null theo cú pháp:

Kiểu_dữ_liệu? tên_biến;

Ngoài ra, Dart cho phép khai báo biến bằng các từ khóa:

var: tự động suy luận kiểu dựa vào giá trị khởi tạo

Cú pháp: `var tên_biến = Giá_trị;`

dynamic (kiểu động, có thể thay đổi kiểu dữ liệu)

Cú pháp: `dynamic tên_biến=Giá_trị;`

Ví dụ: dynamic value = "Hello";//value có kiểu String value=100;//value có kiểu int **Ví dụ:**

```
     void main() {
                  var name = "Flutter"; // Dart suy luận kiểu là String   int age = 25; // Kiểu dữ liệu rõ ràng
     double pi = 3.14;   bool isDartAwesome = true;
                 print("Tên: $name, Tuổi: $age, Pi: $pi, Dart có tuyệt vời không? $isDartAwesome"); }
```

Phạm vi (Scope) của biến

Tương tự như các ngôn ngữ lập trình C/C++, có hai loại phạm vi biến chính trong Dart:

Biến cục bộ (Local Variable)

- Chỉ tồn tại trong một hàm, một khối lệnh ({}).
- Không thể truy cập từ bên ngoài.

Ví dụ:

```
    void main() {
      int number = 10; // Biến cục bộ   if (number > 5) {
        String message = "Số lớn hơn 5";  // Chỉ tồn tại
trong khối if     print(message);
      }
      // print(message);
      //Lỗi: message không tồn tại ở đây
    }
```

Biến toàn cục (Global Variable): Khai báo **ngoài hàm main()**, có thể truy cập ở bất kỳ đâu trong chương trình.

Ví dụ:

```
    String appName = "Dart App"; // Biến toàn cục

    void main() {
      print("Ứng dụng: $appName");
    }
```

1.4.2.3 Hằng

Hằng số là các đại lượng **không thể thay đổi giá trị sau khi được gán**. Dart cung cấp hai từ khóa final và const để định nghĩa hằng số:

final (giá trị được gán một lần trong runtime):

Cú pháp: ***final*** Kiểu_dữ_liệu tên_Hằng = Giá trị;

Ví dụ: final String appName = "Flutter App";

const (giá trị cố định trong compile-time).

Cú pháp: ***const*** Kiểu_Dữ_Liệu tên_Hằng = giá trị;

Ví dụ: const double pi = 3.1415;

Lưu ý:

− Hằng số được khai báo với từ khóa final: được khởi tạo trong runtime (khi chương trình chạy); có thể sử dụng với các giá trị không biết trước khi biên dịch. Ví dụ: final DateTime now = DateTime.now();

− Hằng số được khai báo với từ khóa const: bắt buộc phải có giá trị cố định ngay từ khi biên dịch; không thể nhận các giá trị được khởi tạo từ một hàm hoặc biến động.

1.4.2.4 Toán tử và phép toán

Dart sử dụng các toán tử và phép toán hoàn toàn giống C/C++: +,-,*,/,%,>,>=,<,<=,==,!=, +=,*=,-=,/=,..

1.4.2.5 Cấu trúc điều khiển

Cấu trúc if, switch…case, while, do … while, for hoàn toàn giống ngôn ngữ lập trình C/C++/C#

```
if ( Expression )
{
        Statement1;
}
```

```
if ( Expression )
{
        Statement1;
}
else
{
        Statement2;
}
```

```
if ( Expression_1 ){
        Statement_1;
}
else if(Expression_2){
        Statement_2;
}
…
else{
        Statement_n;
}
```

```
switch ( <biến/biểu thức cần kiểm tra> )
{       case <giá trị 1>:
                <công việc 1>;
        break;          case <giá trị 2>:
                <công việc 2>;
                break;
        …
        [ default:
                <công việc nếu không thuộc trường hợp nào ở trên>; ]
        }
```

```
do {
        Statements;
} while( Expression );
```

```
while(Expression)
{
        Statements;
}
```

```
for (Exp1; Exp2; Exp3)
        Statements;
```

```
for (var value in list)
        Statements;
```

1.5 Bài tập hướng dẫn

Bài tập 1. Cài đặt môi trường lập trình Dart

Để làm quen với ngôn ngữ lập trình Dart, sinh viên có thể sử dụng https://dartpad.dev/ để viết code và biên dịch chương trình.

Phần hướng dẫn cài đặt dưới đây sẽ hướng dẫn thực hiện cài đặt Flutter kèm Dart trong môi trường lập trình Visual Code

Bước 1: Tải Flutter SDK

– Tải Flutter tại https://docs.flutter.dev/get-started/install

– Chọn phiên bản Windows, macOS hoặc Linux phù hợp với hệ điều hành của máy tính

– Tải về và giải nén Flutter SDK vào một thư mục, ví dụ:

- ✓ Windows: C:\flutter
- ✓ macOS: /Users/flutter
- ✓ Linux: /home/flutter

Bước 2: Thêm Flutter vào biến môi trường PATH

– Mở This PC → Properties → Advanced system settings → Environment Variables.

– Trong System Variables, tìm Path, nhấn Edit.

– Nhấn New, thêm đường dẫn C:\flutter\bin.

– Nhấn OK để lưu.

Bước 3: Kiểm tra cài đặt Flutter

– Mở Terminal hoặc CMD và gõ lệnh: *flutter doctor*

– Nếu hiển thị Flutter is installed, nghĩa là cài đặt thành công. ❖ Bước 4: Cài đặt Visual Code

– Tải và cài đặt Visual Code tại đây: https://code.visualstudio.com/download − Mở VS Code → Extensions.

– Cài đặt plugin Flutter và Dart.

Bài tập 2. Làm quen với Dart

Trong cửa sổ Visual Code, nhấn tổ hợp phím **Ctrl + Shift +P.**

Chọn **Dart: New project** ❼ Console Application ❼ chọn thư mục lưu project ->đặt tên project (*my_dart_01*).

Visual code tạo project và mặc định tạo tập tin dart trùng tên project như hình bên dưới.



Chọn view ->terminal ->dart run bin/my_dart_01.dart Kết quả thực thi như hình:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS D:\5.Dev\myprojects\my_dart_01> dart run bin/my_dart_01.dart
Hello world: 42!
PS D:\5.Dev\myprojects\my_dart_01>
```

Thêm một tập tin mới có tên là demo_02.dart vào thư mục ***lib***: Click phải trên thư mục ***lib*** -> new file ->demo_02.dart. Gõ nội dung như hình:

```dart
import 'package:my_dart_01/demo_02.dart';

void main() {
  demo02();
}
```

```dart
import 'dart:io';

bool checkPrime(int x) {
  if (x < 2) return false;
  for (int k = 2; k < x; k++) {
    if (x % k == 0) return false;
  }
  return true;
}
```

Trong thư mục ***bin***, hãy chỉnh sửa nội dung tập tin my_dart_01.dart thành như sau:

```dart
void demo02() {
  stdout.write("Nhập một số nguyên: ");
  String? input = stdin.readLineSync();

  if (input != null && input.isNotEmpty) {
    print("Bạn đã nhập: $input");
    int number = int.parse(input);
    if (checkPrime(number)) {
      print("$number là số nguyên tố.");
    } else {
      print("$number không là số nguyên tố.");
    }
  } else {
    print("Không có dữ liệu nhập vào.");
  }
}
```

Hãy thực thi chương trình trên và cho biết ý nghĩa của đoạn chương trình trên là gì? Dòng lệnh nào để nhập/xuất dữ liệu từ bàn phím?

Trong thư mục **lib** tạo tập tin mới có nội dung như hình bên dưới và gọi thực thi ở tập tin my_dart_01.dart trong thư mục **bin**.

```
void demo03() {
  //tạo một danh sách chứa các số nguyên
  List<int> arrs = [10, 20, 30, 40, 50];
  //xuất ra màn hình các số nguyên trong danh sách theo các cách

  // Cách 1: Sử dụng vòng lặp for
  for (int i = 0; i < arrs.length; i++) {
    print("Phần tử thứ $i: ${arrs[i]}");
  }

  // Cách 2: Sử dụng forEach
  arrs.forEach((num) {
    print("Giá trị: $num");
  });

  // Cách 3: Sử dụng vòng lặp for-in
  for (var num in arrs) {
    print("Số: $num");
  }
}
```

1.6 Bài tập tự làm

Bài tập 1. Hãy viết chương trình cho phép nhập vào một số nguyên >0 là số que kem cần mua, nhập giá tiền cho một que kem. Tính tiền phải trả biết rằng nếu mua số lượng que kem lớn hơn 10 que thì tiền phải trả được giảm 10%; nếu số lượng que kem >=5 và <=10 thì tiền phải trả sẽ được giảm 5%; các trường hợp khác không giảm.

Bài tập 2. Hãy viết chương trình cho phép nhập vào một số nguyên dương > 10.

a. Cho biết số nguyên nhập vào là số có bao nhiêu chữ số?

b. Tính tổng các chữ số có trong số nguyên nhập vào.

c. Cho biết số nhập vào có chứa chữ số nào là số lẻ hay không?

d. Cho biết chữ số lớn nhất trong số đã nhập.

e. Cho biết số nhập vào có chứa chữ số nào nguyên tố hay không?

Bài tập 3. Viết chương trình nhập vào một danh sách các số nguyên và thực hiện các yêu cầu:

a. Xuất danh sách vừa nhập ra màn hình

b. Tính tổng các phần tử có trong danh sách

c. Hãy xuất các phần tử là số nguyên tố có trong danh sách

d. Nhập vào một giá trị bất kỳ, cho biết giá trị đó có trong danh sách hay không? Nếu không có, hãy thêm giá trị đó vào đầu danh sách. Nếu có hãy cho biết giá trị đó ở vị trí nào trong danh sách.

1.7 Bài tập về nhà

Bài tập 1. Hãy viết chương trình cho phép tạo một danh sách các số nguyên ngẫu nhiên trong phạm vi từ 5 đến 100. Thực hiện các yêu cầu sau: a. Hãy xuất các phần tử trong danh sách ra màn hình.

a. Tính tổng các phần tử có trong danh sách.

b. Tính trung bình cộng các số lẻ có trong danh sách. Nếu danh sách không có số lẻ hãy thông báo: Danh sách không có số lẻ.

c. Hãy cho biết danh sách có là danh sách đối xứng hay không?

d. Hãy cho biết danh sách tạo ra có được sắp xếp tăng dần hay không?

e. Tìm phần tử lớn nhất có trong danh sách.

f. Tìm phần tử là số chẵn lớn nhất. Nếu danh sách không có số chẵn hãy thông báo danh sách h ông có số chẵn.

g. Hãy nhập một giá trị, tìm xem giá trị vừa nhập có trong danh sách hay không? Nếu danh sách không có giá trị vừa nhập, hãy thông báo: Không tìm thấy. Nếu tìm thấy, hãy xóa các phần tử có cùng giá trị với giá trị vừa nhập.

Bài tập 2. Hãy viết chương trình thực hiện các yêu cầu sau:

a. Nhập vào 1 chuỗi và xuất chuỗi đó ra màn hình

b. Cho biết chuỗi có bao nhiêu kí tự là nguyên âm?

c. Cho biết chuỗi có bao nhiêu từ?

d. Cho biết chuỗi có đối xứng hay không?

e. Đảo ngược từ trong chuỗi. Ví dụ: Bò ăn cỏ, cỏ ăn Bò.